TÜRK HAVA KURUMU ÜNİVERSİTESİ
UNIVERSİTY OF TURKISH AERONAUTICAL ASSOCIATION



2025 - 2026 SPRING

Computer Engineering Graduation Design II

Autonomous Car with Reinforcement Learning

Project Short Title	RL-AutonomousCar
Project Full Title	Autonomous Car with Reinforcement Learning
Subject Classification	Reinforcement Learning / Autonomous Systems
Student name	Mehmet Efe KAPISIZ - 210444029
Supervisor	Ziya KARAKAYA

Abstract

Rule-based artificial intelligence systems have long dominated video game and simulation environments for autonomous vehicle behavior. These systems rely on handcrafted logic trees and deterministic path-finding algorithms, which are rigid by nature: they cannot adapt to unforeseen scenarios, require manual rule updates for each new environment configuration and fail to generalize when dynamic elements are introduced. This limitation motivates the use of Reinforcement Learning (RL), a paradigm in which an agent learns an optimal behavior policy through trial-and-error interaction with its environment, without requiring explicit programming of every possible situation.

This project implements and evaluates an RL-based autonomous car agent within the Unity ML-Agents framework. The agent navigates a winding road, avoids dynamic obstacles, and passes through a sequence of ordered waypoints before crossing a finish line. To enable environmental perception, the agent is equipped with a Ray Perception Sensor that simulates LiDAR-like distance measurements for obstacle detection and navigation. Two training algorithms are systematically compared under identical environment conditions: Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC). Then a five-stage Curriculum Learning approach using PPO with progressively increasing obstacle difficulty.

The originality of this work lies in the extreme training conditions imposed by environmental design. The combination of dynamic obstacles and ray perception sensors with limited field of view creates a scenario in which the agent cannot see beyond the next waypoint, leaving minimal reaction time when an obstacle enters its detection range. This forces the agent to develop fast, decisive maneuvers under high environmental uncertainty rather than relying on long-range planning. Unlike prior works that evaluate RL-based navigation in static or low-variance environments, this project deliberately maximizes environmental stochasticity. Training under these conditions requires the agent to learn a genuinely robust and generalizable policy, rather than memorizing a fixed sequence of actions that happens to work in a predictable environment.

Keywords: Reinforcement Learning, Proximal Policy Optimization, Soft Actor-Critic, Curriculum Learning, Unity ML-Agents

Acknowledgements

I would like to express my sincere gratitude to my project supervisor for the guidance and constructive feedback provided throughout the development of this project. I also acknowledge the open-source contributions of the Unity ML-Agents team at Unity Technologies and Google DeepMind, whose frameworks made this research possible. Finally, I thank the University of Turkish Aeronautical Association for providing the academic environment and resources that supported this work.

Table of Contents

Abstract
Keywords: Reinforcement Learning, Proximal Policy Optimization, Soft Actor-Critic, Curriculum Learning, Unity ML-Agents
Acknowledgements
Ch1: Introduction
1.1 Problem Statement
1.2 Scope
Ch2: Project Plan
2.1 Methodology
2.2. Time Schedule (Gantt Chart)
2.3. Deliverables List
Ch3: Literature Review
3.1 Overview and Scope
3.2 Simulation Platforms and the Role of Unity ML-Agents
3.3 Reinforcement Learning Approaches for Autonomous Navigation
3.3.1 Imitation Learning and Human-Guided Approaches
3.3.2 Algorithm Comparison and Reward Function Design
3.4 Curriculum Learning and Progressive Training Design
3.5 Conclusion of the Literature Review
Ch4: Requirements
4.1 Overall Description of the Project
4.1.1 Product Perspective
4.1.2 Product Functions
4.1.3 User Characteristics
4.1.4 Constraints
4.1.5 Assumptions and Dependencies
4.2 Specific Requirements
4.2.1 External Interface Requirements
4.2.2 Functional requirements
4.2.3 Quality Requirement
Ch5: Design
5.1 Software Architecture

5.1.1 Actors
5.1.2 Constraints
5.1.3 Software Architecture
5.2 Views
5.2.1 Logical View
5.2.2 Process View
Ch6: Implementation & Test
6.1 Implementation
6.1.1 Stage 1: Initial Prototype
6.1.2 Stage 2: Track Extension and Waypoint System
6.1.3 Stage 3: Obstacle Integration and Observation Space Redesign
6.1.4 Stage 4: Visual Polish and UI
6.1.5 Stage 5: Final Training Pipeline
6.2 Test
6.2.1. Training Monitoring (Continuous)
6.2.2 In-Distribution Inference Test
6.2.3 Out-of-Distribution Generalization
6.2.4 Performance Test
Ch7: Evaluation
7.1 Understanding the Metrics
7.2 Curriculum Learning Results (CurriculumRun1 to Run5)
7.3 PPO_Run1 vs. CurriculumRun5
7.4 PPO_Run1 vs. SAC_Run1
7.5 Summary
Ch8: Impact of the Project & Compliance with the Constraints
8.1 Compliance with the Realistic Constraints
8.2 Impact of the Project
Ch9: Conclusions
Appendix A
Appendix B
References

Acronyms

Acronym	Definition
RL	Reinforcement Learning
PPO	Proximal Policy Optimization
SAC	Soft Actor-Critic
ML	Machine Learning
AI	Artificial Intelligence
CUDA	Compute Unified Device Architecture
GPU	Graphics Processing Unit
MDP	Markov Decision Process
UE	Unreal Engine
LiDAR	Light Detection and Ranging
RPS	Ray Perception Sensor
UI	User Interface
API	Application Programming Interface

Ch1: Introduction

1.1 Problem Statement

Autonomous vehicle behavior in simulated environments has traditionally been implemented through rule-based systems: finite state machines, scripted patrol paths, and etc. While these approaches are predictable and easy to debug, they are fundamentally brittle. A rule-based agent cannot adapt to conditions its designer did not explicitly anticipate. When dynamic elements are introduced (such as moving obstacles with varying speeds) the system requires manual reconfiguration for every new scenario. This lack of adaptability becomes a critical limitation in environments where conditions change continuously and unpredictably.

To address this, the current study moves beyond static, rule-based frameworks by leveraging Reinforcement Learning (RL), specifically comparing Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) algorithms. However, traditional RL implementations often rely on long-range planning or perfect environmental information. The core problem lies in developing an agent capable of high-speed navigation under extreme environmental uncertainty, where the agent's visibility is restricted to local ray-perception inputs, effectively forcing reactive decision-making over long-term path memorization.

Furthermore, the lack of robust training strategies for highly stochastic environments remains a significant bottleneck in current research. Standard training methodologies often fail to generalize when the agent encounters dynamic obstacles within limited detection ranges. By implementing a curriculum learning approach this project aims to bridge the gap between simple simulation navigation and the need for agents that can handle genuine environmental complexity and unpredictable obstacle interactions, rather than merely memorizing fixed sequences of actions.

1.2 Scope

This project delivers a simulation-based autonomous car agent trained entirely within the Unity ML-Agents framework. The application is a custom-built winding road environment in which the agent must learn to navigate through waypoints and cross a finish line while avoiding both static and dynamic obstacles, without any handcrafted movement logic.

The primary objective is to determine which RL training strategy produces the most robust and generalizable navigation policy in a dynamic obstacle environment. To this end, three strategies are implemented and evaluated under controlled comparable conditions: PPO

trained from scratch, SAC, and a five-stage Curriculum Learning pipeline using PPO with progressively increasing obstacle speeds.

The key benefits of the developed system are as follows. First, the agent's behavior emerges entirely from interaction with the environment. Second, the curriculum-based training pipeline is designed to manage environmental stochasticity by progressively increasing complexity, thereby reducing variance and improving convergence stability during the learning process. Third, comparative evaluation across algorithms provides actionable insight into which training strategy is best suited to dynamic simulation environments.

The scope is intentionally bound to simulation. Real hardware deployment, multi-agent competition, and sensor fusion are outside the scope of this project. The environment runs locally on a single workstation, and all trained models are evaluated within Unity in inference mode.

Ch2: Project Plan

2.1 Methodology

This project employs an iterative, experiment-driven development methodology, structured to accommodate the exploratory nature of Reinforcement Learning (RL) research. Given the unpredictability inherent in training autonomous agents, a rigid sequential approach was deemed inappropriate. Instead, development proceeded through cyclical phases of design, implementation, training, and empirical evaluation, where findings from each cycle directly informed the subsequent iteration.

The process originated with the establishment of a minimal working prototype, consisting of a simplified environment with a basic reward structure, which was subsequently expanded in incremental stages. Each iteration focused on isolating variables, such as introducing dynamic obstacles, modifying the ray-perception sensors, or adjusting hyperparameter configurations. When iterations yielded suboptimal performance, these were treated not as failures, but as diagnostic data points to refine the environment and training configuration.

This methodological framework aligns with the principles of Rapid Application Development (RAD), prioritizing the iterative validation of design decisions through direct experimentation over static, upfront specifications. By functioning as a continuous feedback loop, this approach allowed for the systematic optimization of the RL agent's policy, facilitating a transition from basic navigation to robust, autonomous behavior in high-variance environments.

2.2. Time Schedule (Gantt Chart)

WP / Task No	WP / Task Name	Start Date	End Date	Semester for Weeks																	
				1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
T1	Environment Setup & Tool Configuration	23 February	8 March		■	■															
T2	Road & Waypoint Design, Basic Agent	9 March	15 March				■														
T3	Reward Function	16 March	22 March					■													
T4	Obstacle Integration & Sensor Configuration	23 March	29 March						■												
T5	Visual Polish, Asset Integration	30 March	5 April							■											
T6	Curriculum Learning Training	6 April	12 April								■										
T7	PPO, SAC & Parallel Training & Generalization Test	13 April	26 April									■	■								
T8	Report Writing & Finalization	27 April	10 May											■	■						

2.3. Deliverables List

ID	Type	Description	Due Date
D1	Document	Spring Semester Final Report (CENG 496)	June 2026
D2	Software	Unity project with trained ML-Agents models	June 2026
D3	Other	Poster Presentation	June 2026

Ch3: Literature Review

3.1 Overview and Scope

The application of reinforcement learning (RL) to autonomous vehicle navigation in simulated environments has emerged as an active and rapidly evolving area of research. As simulation platforms more sophisticated, researchers have increasingly turned to RL-based approaches to develop agents capable of navigating complex and dynamic scenarios without hand-crafted rules. This chapter reviews the most relevant prior work in this domain and situates the present project within the existing literature, identifying both the foundations upon which it builds and the specific contributions it makes.

3.2 Simulation Platforms and the Role of Unity ML-Agents

A critical thing of modern RL-based autonomous navigation research is the availability of high-fidelity simulation environments. Among these, Unity has emerged as a widely adopted platform due to its flexible physics engine, visual fidelity, and the Unity ML-Agents toolkit, which provides native integration between the simulation environment and standard RL training pipelines. Several recent studies have validated Unity as a viable and accessible platform for this class of research.

Raut, Galchhaniya, Nehete and Bhoite (2024) provided a broad overview of how RL is reshaping game AI through the Unity ML-Agents toolkit, arguing that RL enables adaptive agent behavior that rigid, rule-based systems cannot achieve. Sung and Kim (2025) provide a more targeted demonstration of Unity ML-Agents in an autonomous navigation setting, training a kart agent to follow sequential waypoints on track while avoiding static obstacles. Their primary focus is reward function design, examining how different reward shaping strategies influence agent behavior and convergence. The present project builds on this foundation while extending the experimental scope to include algorithm comparison, curriculum-based training design, and out-of-distribution generalization testing.

3.3 Reinforcement Learning Approaches for Autonomous Navigation

3.3.1 Imitation Learning and Human-Guided Approaches

One prominent line of research has sought to produce naturalistic driving behavior by incorporating human expert knowledge directly into the training process. Emuna, Borowsky

and Biess (2020) investigated deep reinforcement learning for human-like driving behavior in collision avoidance scenarios using Unity as the simulation environment. Their approach combined a model-driven reward component encoding basic driving rules (lane-keeping, speed control) with a data-driven component derived from the track position and speed distributions of an experienced human driver. The agent's policy was parameterized as a Mixture Density Network (MDN) atop the Proximal Policy Optimization (PPO) algorithm, enabling the reproduction of both the mean and variance of expert behavior. Notably, the authors first attempted Deep Deterministic Policy Gradient (DDPG) but abandoned it due to instability despite extensive hyperparameter tuning, ultimately selecting PPO for its more reliable convergence properties.

While the work of Emuna et al. (2020) shares the use of Unity and the obstacle avoidance objective with the present project, two key differences distinguish the approaches. First, Emuna et al. rely on imitation learning from collected human driving data, whereas the present project trains entirely through environmental interaction with no human demonstrations, reducing dependency on data collection and expert availability. Second, the obstacle configuration in their study is static, with seventeen fixed barricades placed along a closed highway track, whereas the present project introduces dynamic obstacles with varying speeds, representing a substantially harder generalization challenge that more closely approximates real-world driving conditions.

3.3.2 Algorithm Comparison and Reward Function Design

Perhaps the closest existing work to the present project in both scope and methodology is the comparative study conducted by Savid, Mahmoudi, Maskeliunas and Damasevičius (2023), which evaluated multiple RL algorithms for simulated autonomous driving using the Unity ML-Agents toolkit. Training kart agents to navigate a racing track and avoid obstacles, the study found that PPO yielded the best results among the algorithms tested, achieving a value loss of 0.0013 and a cumulative reward of 0.761. To handle obstacle avoidance, a separate model was trained with Behavioral Cloning (BC) as a pre-training strategy, which achieved a value loss of 0.001 and a cumulative reward of 0.068. The authors also compared Multi-Agent PPO (MA-PPO) and POCA (Parallel Online Continuous Arcing) configurations, establishing PPO as the most reliable baseline for this class of tasks.

The present project diverges from Savid et al. (2023) in two important respects. First, rather than Behavioral Cloning, this project employs a five-stage Curriculum Learning pipeline to progressively introduce obstacle difficulty, avoiding any dependency on expert demonstrations and enabling the agent to develop robust behavior through structured environmental exposure alone. Second, this project explicitly compares PPO and Soft Actor-Critic (SAC) under identical experimental conditions, which is not addressed by Savid et al. (2023). SAC is selected as the comparative algorithm due to its maximum-entropy objective, which encourages broader policy exploration and has demonstrated competitive performance in continuous control tasks (Haarnoja et al., 2018).

3.4 Curriculum Learning and Progressive Training Design

The use of curriculum learning (Bengio et al., 2009), structuring training so that an agent encounters progressively more difficult scenarios, has gained traction as a principled approach to accelerating RL convergence and improving generalization. By gradually increasing task complexity, curriculum learning prevents early convergence to suboptimal policies and ensures that foundational behaviors are established before harder challenges are introduced.

In the context of obstacle avoidance, the benefit of progressive difficulty introduction is evident when comparing the two phases of Savid et al. (2023): the obstacle avoidance model, introduced abruptly as a separate training phase with Behavioral Cloning pre-training, achieved substantially lower cumulative rewards than the baseline track-following model, suggesting that obstacle difficulty was not introduced in a manner that allowed the agent to build on previously acquired skills. The present project addresses this directly through a five-stage curriculum that begins with an open track and culminates in dense, dynamically moving obstacles, allowing the agent to consolidate lane-following and speed management behaviors before confronting the full complexity of the task.

3.5 Conclusion of the Literature Review

The reviewed literature establishes several important foundations for the present project. First, PPO is a strong and well-validated baseline for simulated autonomous driving tasks in Unity, demonstrating superior convergence stability over alternatives such as DDPG (Emuna et al., 2020) and competitive performance against multi-agent variants (Savid et al.,

2023). Second, both the choice of algorithm and the design of the reward function exert decisive influence over agent behavior, as demonstrated by Savid et al. (2023). Third, obstacle avoidance remains a challenging open problem that benefits significantly from careful training design, particularly when obstacles are dynamic rather than static (Emuna et al., 2020). Fourth, Unity ML-Agents is a well-supported and accessible platform for this class of research (Savid et al., 2023; Sung & Kim, 2025; Raut et al., 2024). Fifth and most notably, existing research often evaluates performance in static or low-variance settings, failing to address the agent's ability to maintain high-speed, decisive navigation under extreme environmental stochasticity.

Ch4: Requirements

4.1 Overall Description of the Project

4.1.1 Product Perspective

This product is an independent, self-contained simulation system. It does not function as a component of a larger external platform and has no dependencies on third-party services or networked systems at runtime. The system consists of two loosely coupled components that interact locally: the Unity simulation environment and the Python ML-Agents training process. During training, Unity acts as the environment client and communicates with the Python trainer via a local gRPC socket. During inference, the trained ONNX model is loaded directly into Unity, eliminating the Python dependency entirely. No external APIs, databases, or cloud services are involved.

4.1.2 Product Functions

The system performs the following major functions:

- Simulates a winding road environment with dynamic obstacles, ordered waypoints, and a finish line.
- Trains a car agent using RL algorithms (such as PPO, SAC) through interaction with the simulation.
- Collects observations from Ray Perception Sensors and vector inputs at each decision step and passes actions back to the agent.
- Applies a shaped reward function to guide the agent toward completing the course while avoiding collisions.
- Exports training metrics to TensorBoard for real-time and post-training analysis.
- Runs trained models in inference mode to demonstrate and evaluate learned behavior.
- Displays real-time statistics (episode count, speed, reward, laps, collisions) via an in-scene UI.

4.1.3 User Characteristics

The primary user of this system is the developer, a final-year Computer Engineering student with experience in Unity, C#, and Python. Interaction with the system requires familiarity with the Unity Editor, the ML-Agents Python CLI, and basic command-line usage.

The secondary audience is the academic jury evaluating the project, who interact with the system passively through the poster presentation, this report, and the inference-mode demonstration. No end-user interface for non-technical users is provided or required.

4.1.4 Constraints

- Training is limited to a single local workstation; distributed or cloud-based training is not supported.
- The ML-Agents Python package version (1.1.0) supports only PPO and SAC.
- PyTorch 2.2.1 with CUDA 12.1 is required; newer versions were not tested and may introduce compatibility issues.
- The simulation environment is custom-built and not based on any standardized autonomous driving benchmark, limiting direct comparison with published results.

4.1.5 Assumptions and Dependencies

- It is assumed that Unity 6.3 LTS, Python 3.10.12, and the specified ML-Agents versions are correctly installed and configured.
- The system assumes a CUDA-capable GPU is available; CPU-only training is theoretically possible but was not tested and would be prohibitively slow.
- It is assumed that the gRPC communication between Unity and the Python trainer remains stable throughout a training session; prolonged disconnections would terminate training.

4.2 Specific Requirements

4.2.1 External Interface Requirements

The system has the following inputs and outputs:

- **Input - Vector Observations**
  - **Purpose:** Provide the agent with self-state and navigation information at each decision step.
  - **Source:** CarAgent script (Unity).
  - **Values:** 8 continuous floating-point numbers (velocity.x, velocity.z, forward.x, forward.z, target direction x, target direction z, dot product, normalised distance).

- **Timing:** collected every 5 simulation steps (Decision Period = 5).
- **Format:** Array passed via gRPC to the Python trainer.

- **Input - Ray Perception Sensor Data**
- **Purpose:** Provide spatial awareness of surrounding objects.
- **Source:** Two RayPerceptionSensor3D components attached to the agent.
- **Values:** per-ray hit distance and one-hot tag encoding
- **Timing:** sampled at every decision step alongside vector observations.

- **Output - Action Vector**
- **Purpose:** Control the agent's movement.
- **Destination:** CarAgent script (Unity physics engine)
- **Values:** 2 continuous floats in the range [−1.0, +1.0]. Action[0]: throttle (positive) or brake (negative) Action[1]: steering direction.
- **Timing:** one action vector returned per decision step.

- **Output - TensorBoard Logs**
- **Purpose:** Record training progress for analysis.
- **Format:** TensorBoard event files.
- **Content:** Mean Reward, Std of Reward, policy loss, value loss, per 5,000 steps.

- **Output - ONNX Model Checkpoint**
- **Purpose:** Persist the trained policy for inference.
- **Destination:** 'results' directory.
- **Format:** .onnx file. Updated periodically during training and at run completion.

- **Output - Unity UI Display**
- **Purpose:** Real-time training monitoring.
- **Destination:** Canvas TextMeshPro elements visible in the Unity Game view.
- **Content:** Episode count, Laps, Collisions, Speed, Cumulative Reward, Best Reward.
- **Timing:** updated every frame.

4.2.2 Functional requirements

The system shall collect 8 continuous vector observations and full ray perception sensor data at every decision step and transmit them to the Python trainer via gRPC.

The system shall receive a 2-dimensional continuous action vector from the trainer and apply it to the agent's Rigidbody: positive Action[0] applies forward acceleration force, negative Action[0] applies braking force, and Action[1] rotates the agent about the Y-axis.

The system shall advance the active waypoint target index upon trigger collision with each Waypoint object in sequential order (Waypoint1 → Waypoint2 → ... → Waypoint5 → Finish). The agent shall not be able to skip waypoints out of order.

The system shall apply a distance-based shaping reward of $+\Delta d \times 0.1$ at each decision step, where Δd is the reduction in distance to the current target since the previous step.
The system shall apply a milestone reward of +0.3 upon passing each Waypoint trigger and +2.0 upon crossing the Finish trigger, then call EndEpisode().

The system shall apply a penalty of −0.5 and call EndEpisode() upon collision with any Wall-tagged object, and a penalty of −1.0 and call EndEpisode() upon collision with any Obstacle-tagged object.

The system shall reset the agent's position, rotation, velocity, and target index to their initial values at the start of each new episode.

The system shall update all UI elements (Episode, Laps, Collisions, Speed, Reward, Best Reward) every frame during training.

In the event of a gRPC communication failure between Unity and the Python trainer, the Unity environment shall freeze at the current state and await reconnection; no automatic recovery or data loss prevention mechanism is implemented beyond ML-Agents's default behavior.

4.2.3 Quality Requirement

Hardware: Tested on RTX 3080 Ti (12 GB VRAM), Intel Core i9-11900K, 32 GB DDR4 RAM.

Operating System: Windows 11 (64-bit). No cross-platform testing was performed.

Programming Environment: Unity 6.3 LTS (6000.3.11f1), ML-Agents Unity package 4.0.2, Python 3.10.12, ML-Agents Python package 1.1.0, PyTorch 2.2.1 with CUDA 12.1, Visual Studio 2022.

Performance: PPO single environment: ~58 minutes for 500,000; SAC: ~6.5 hours for same step count

Security: No user data is collected or transmitted. All communication occurs locally over gRPC on localhost. No authentication or encryption is required.

Reliability: Training sessions may be interrupted and resumed using ML-Agents's checkpoint mechanism. The system does not implement automatic fault

Ch5: Design

5.1 Software Architecture

5.1.1 Actors

CarAgent: The primary actor. A C# class inheriting from Unity ML-Agents's Agent base class. Responsible for collecting observations, receiving action vectors, applying physics forces to the Rigidbody, calculating and assigning rewards, and managing episode lifecycle.

ML-Agents Python Trainer: An external process launched via the ML-Agents CLI. Receives observations from Unity, updates the neural network policy (PPO or SAC), and returns action vectors. Writes model checkpoints (.onnx) and TensorBoard event files to the local results directory.

ObstacleMovement Script: A C# component attached to each of the five obstacle objects. Controls obstacle patrol behaviour; when moveSpeed is set to zero the obstacle is static, when greater than zero it follows a predefined movement path at the specified speed.

WheelRotation Script: A C# component attached to the car. Rotates the four wheel meshes each frame proportionally to the agent's current forward speed, providing visual feedback without affecting physics.

CarUI Script: A C# component that reads CarAgent's public fields each frame and updates the Canvas TextMeshPro elements displaying Episode, Laps, Collisions, Speed, Cumulative Reward, and Best Reward.

TensorBoard: A passive monitoring actor. Reads event files written by the Python trainer and serves visualizations at localhost:6006.

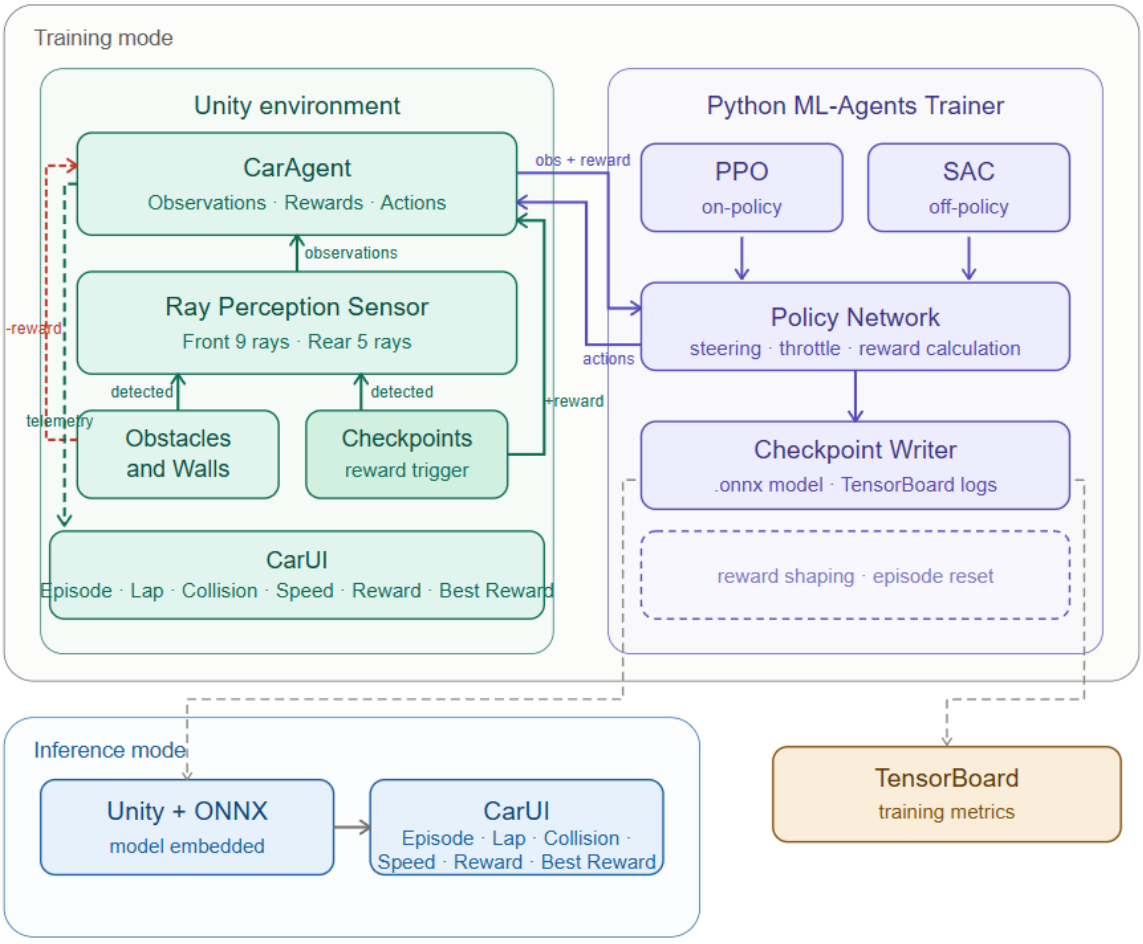
5.1.2 Constraints

- The gRPC communication between Unity and the Python trainer operates over localhost only; remote training across machines is not supported.
- The ML-Agents Python package version 1.1.0 restricts only available algorithms to PPO and SAC.
- The neural network policy is a fully connected feedforward network; recurrent architectures (LSTM) were not used, meaning the agent has no memory across decision steps beyond what is encoded in the current observation vector.

- Physics simulation is handled by Unity's built-in Rigidbody system with simplified vehicle dynamics; no tyre friction model, suspension, or weight transfer is implemented.
- The Decision Period of 5 means the agent acts at one-fifth the simulation frame rate; the action is held constant between decisions, which reduces training complexity but limits reaction speed.

5.1.3 Software Architecture

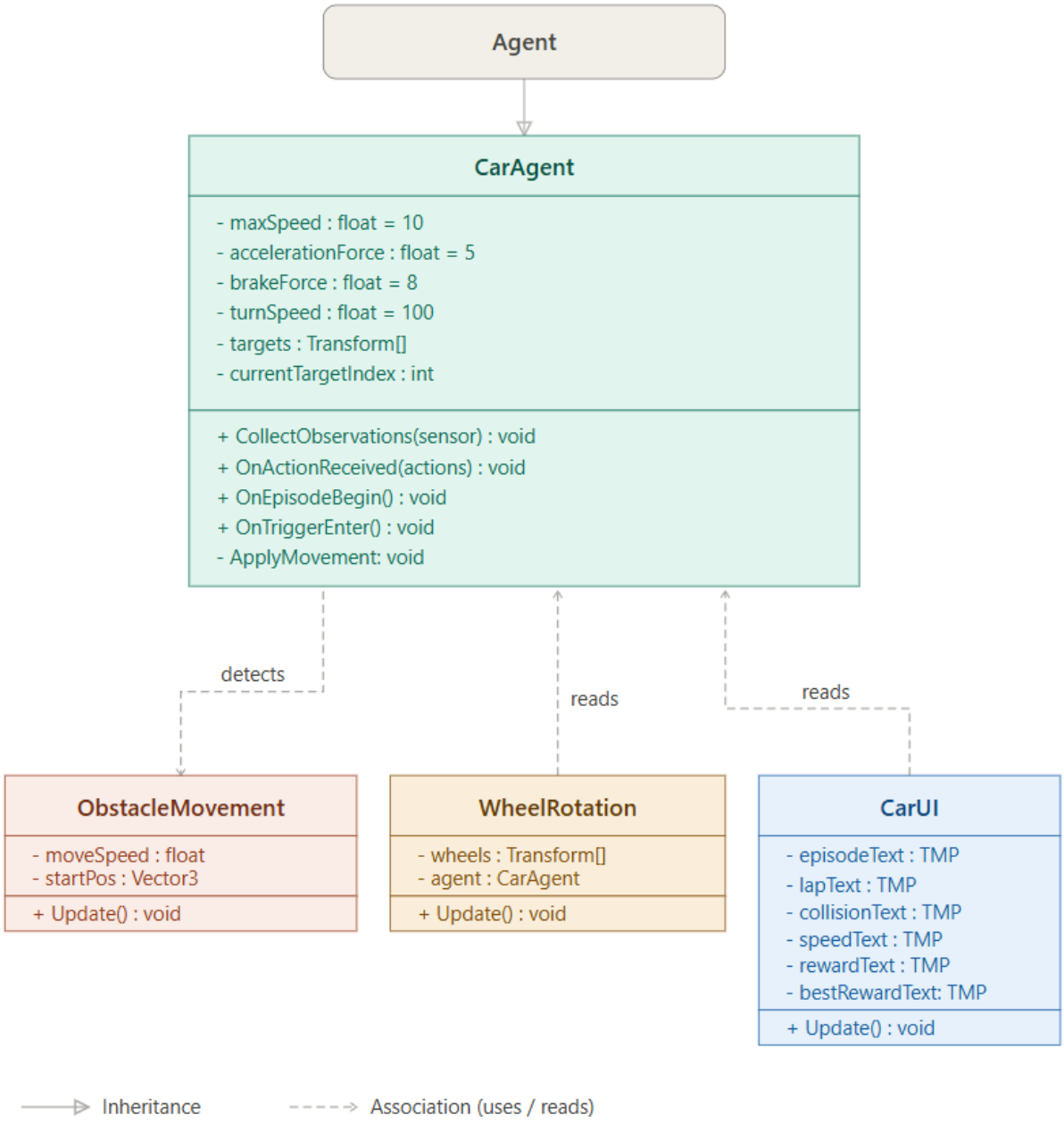
Component Diagram



5.2 Views

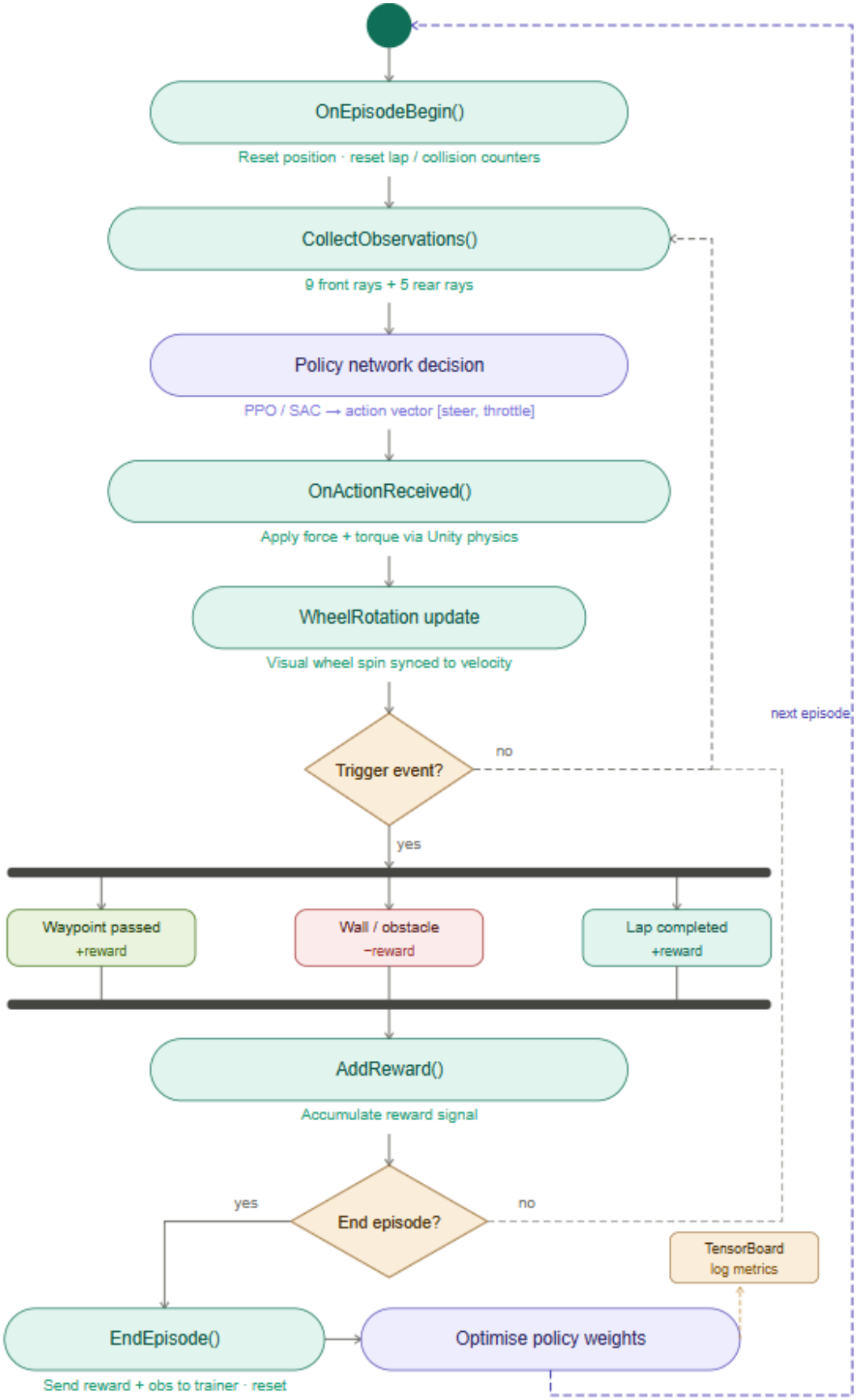
5.2.1 Logical View

Class Diagram

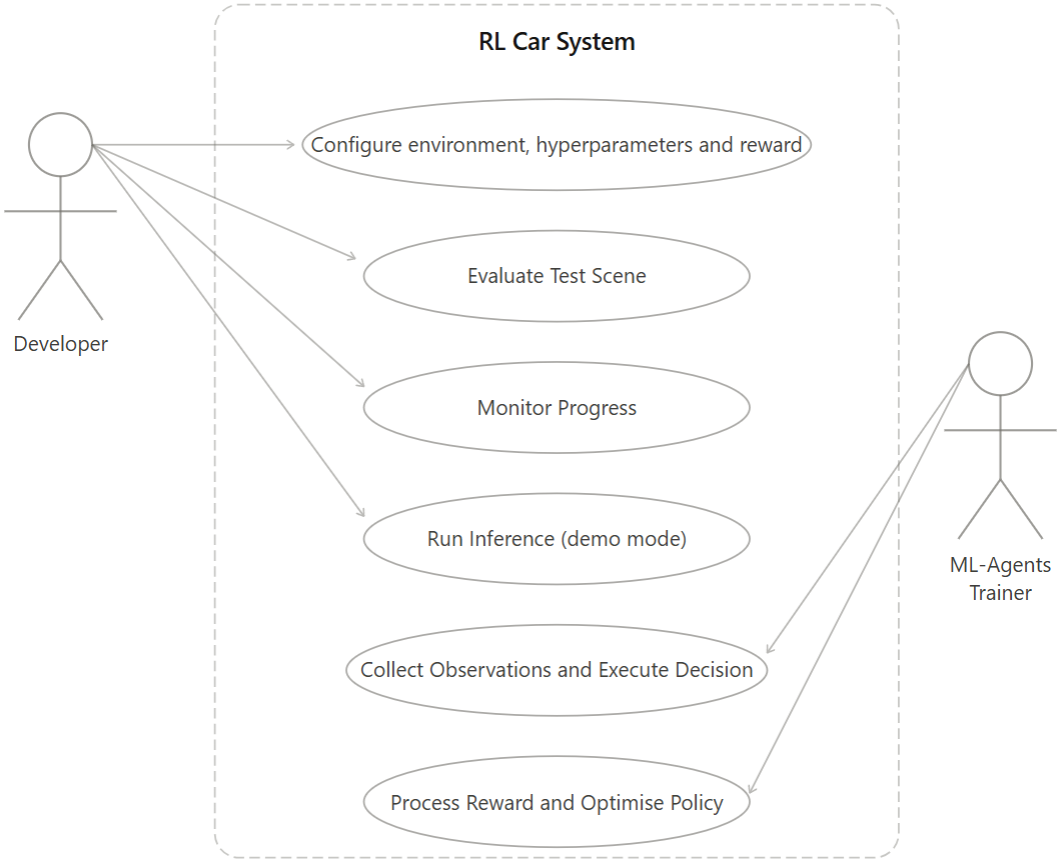


5.2.2 Process View

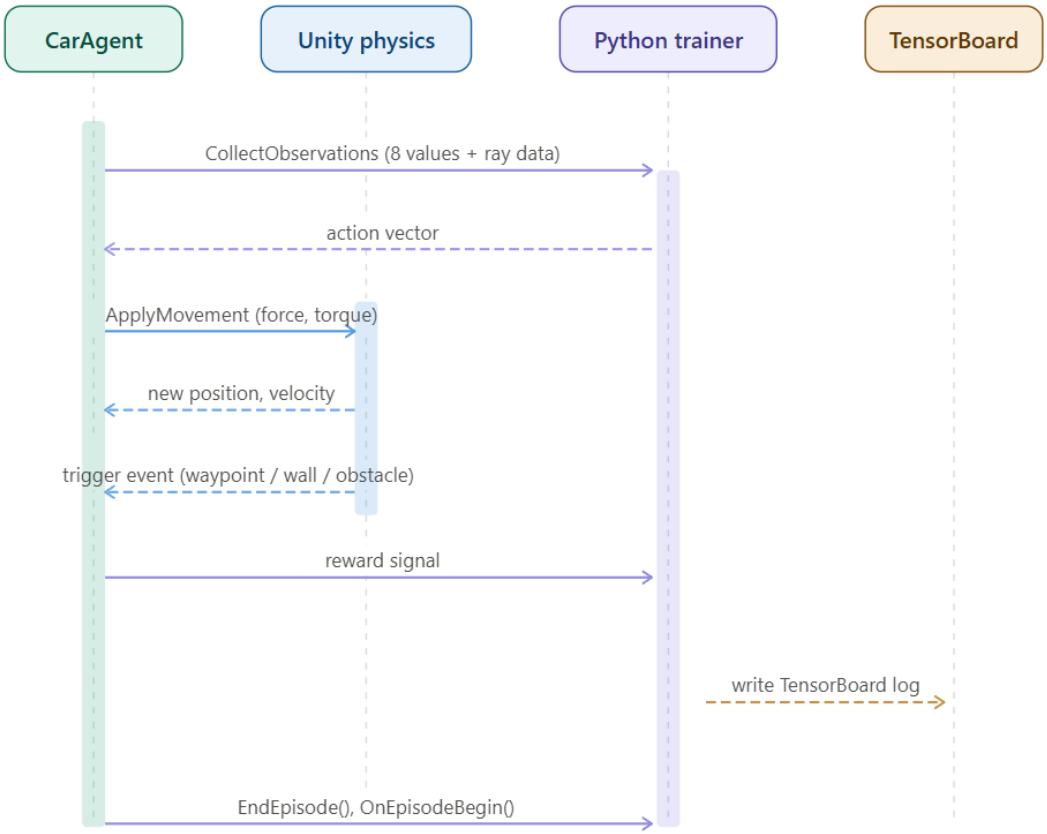
Activity Diagram



Use Case Diagram



Sequence Diagram

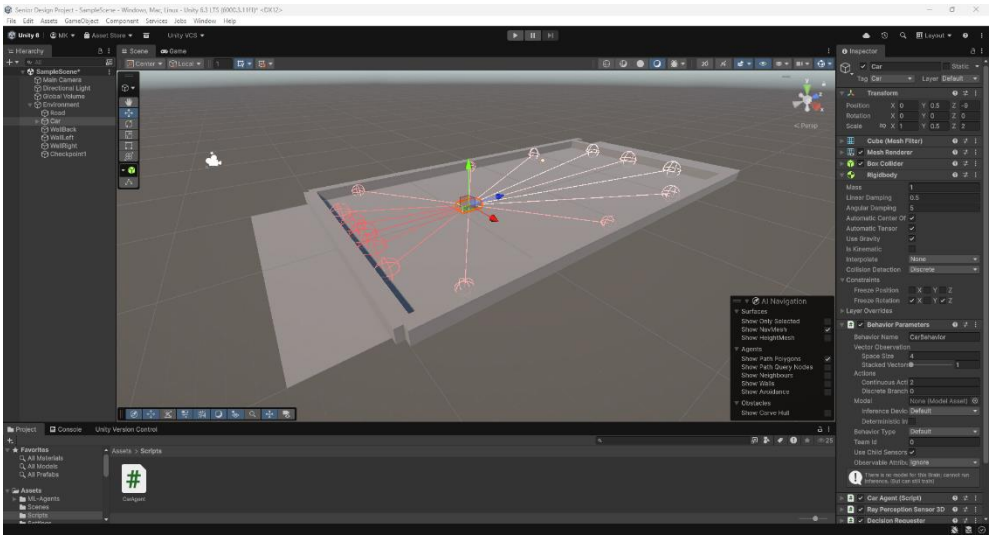


Ch6: Implementation & Test

6.1 Implementation

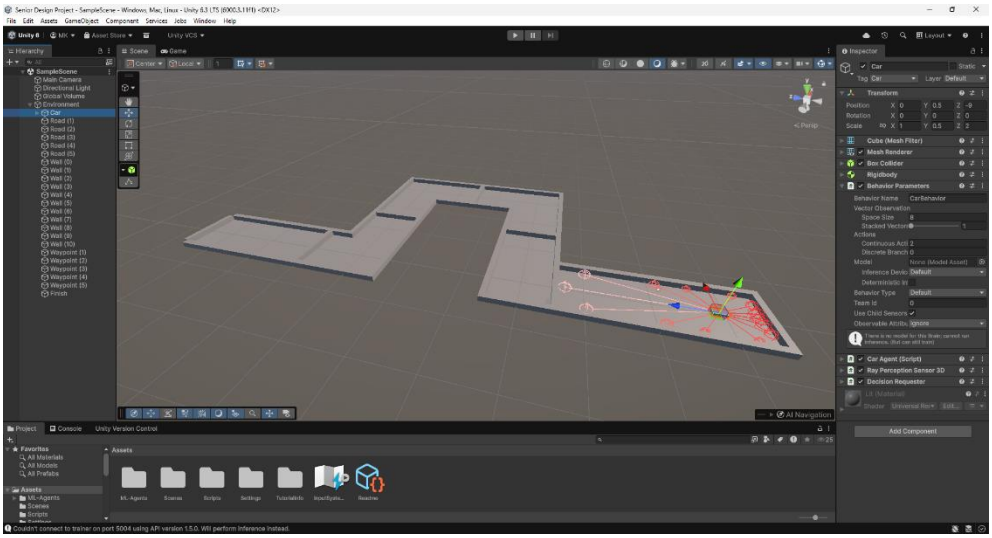
6.1.1 Stage 1: Initial Prototype

The first working prototype was built in Unity 6.3 LTS with the ML-Agents 4.0.2 package. A simple scene was constructed consisting of a flat straight road, boundary walls, and a single checkpoint trigger. The CarAgent script was written with a basic observation space and reward function, and PPO training was initiated. Within approximately 20,000 training steps the agent successfully reached the checkpoint, achieving a Mean Reward of approximately 3.75 with a Std of Reward of 0.017, indicating stable but elementary behavior.



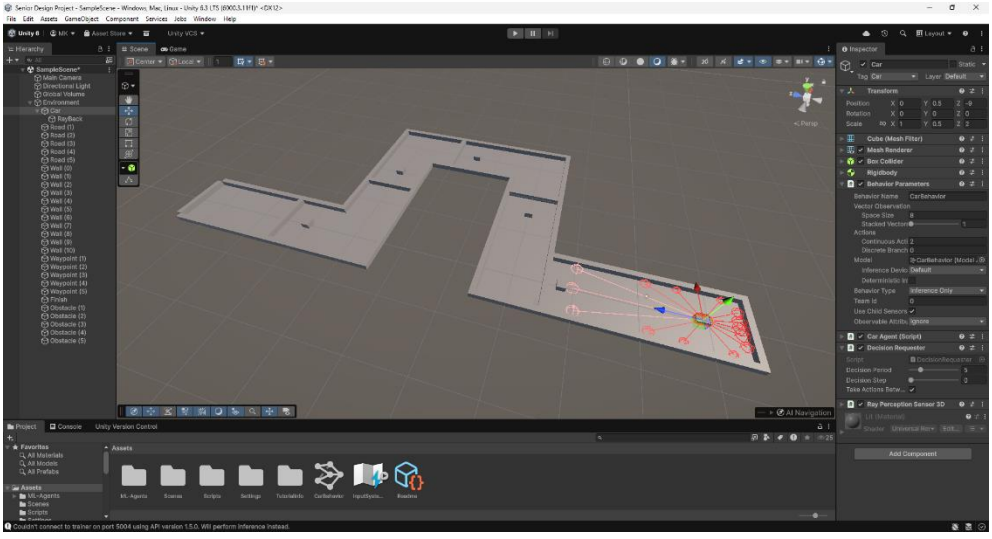
6.1.2 Stage 2: Track Extension and Waypoint System

The road was extended and redesigned as a winding course requiring the agent to learn steering behavior in addition to forward progress. Five ordered waypoint triggers and a finish line trigger were added to define the navigation objective. Training on this extended environment produced a model that navigated the full winding track in inference mode without intervention, validating the observation space design and the distance-based reward shaping approach.



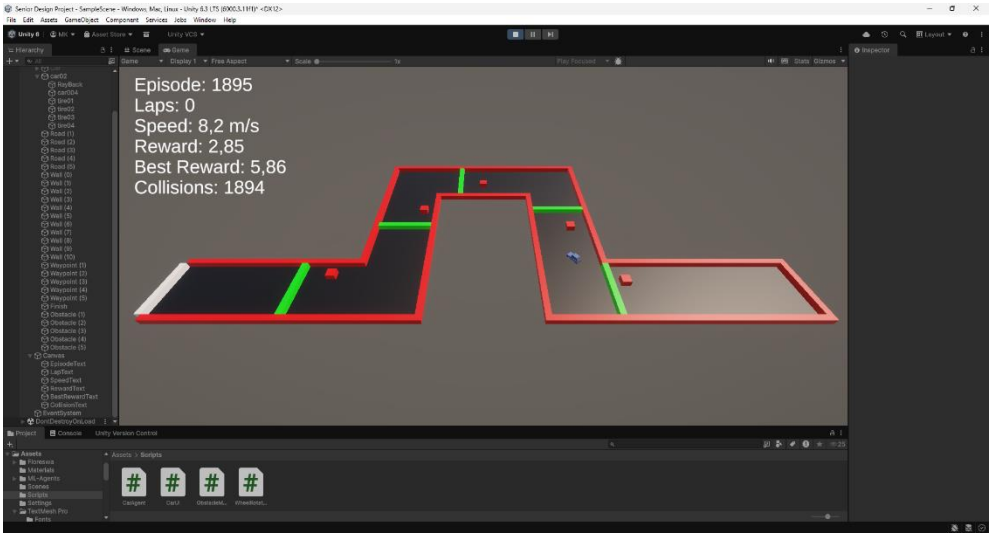
6.1.3 Stage 3: Obstacle Integration and Observation Space Redesign

Five obstacle objects were introduced to the scene, each controlled by the ObstacleMovement script. An initial attempt to fine-tune the existing trained model on the new obstacle environment failed due to a structural incompatibility: the previous model had been trained with a different ray sensor configuration, resulting in a mismatched observation space dimension. This is distinct from catastrophic forgetting; it is a hard architectural mismatch that prevents weight transfer entirely. The solution was to redesign the sensor configuration first (finalizing the front and rear RayPerceptionSensor3D parameters), then retrain from scratch with the correct architecture. A first curriculum learning sequence was subsequently conducted, progressing from no obstacles to static obstacles to slow-moving obstacles, producing a functional obstacle-aware model.



6.1.4 Stage 4: Visual Polish and UI

With the core functionality validated, visual assets from the Unity Asset Store were applied to the environment and car model. The WheelRotation script was implemented to animate the wheel meshes proportionally to the vehicle's forward speed. A Canvas UI was added displaying Episode count, Laps, Collisions, Speed, Cumulative Reward, and Best Reward, updated every frame by the CarUI script. Crucially, scene rendering was kept deliberately minimal throughout training to maximize simulation throughput and reduce per-frame GPU overhead during the training runs.

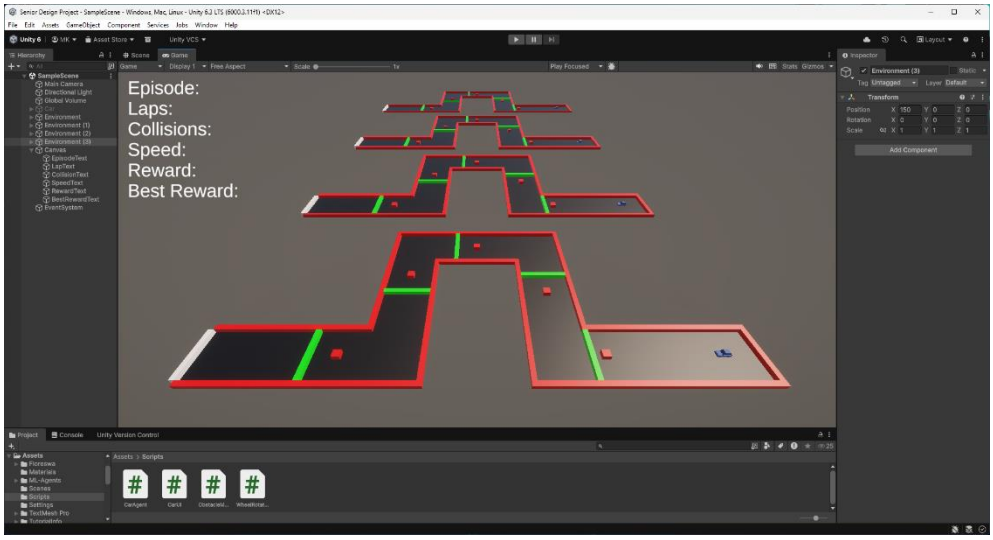


6.1.5 Stage 5: Final Training Pipeline

The complete final training sequence was conducted following the sensor configuration changes that invalidated the earlier curriculum models. This comprised five curriculum stages (CurriculumRun1-5), one PPO single-run baseline (PPO_Run1), and one SAC run (SAC_Run1).

Additionally, a parallel training experiment using four simultaneous environments was initiated to evaluate potential improvements in convergence efficiency. For this purpose, the CarAgent script was extended to ensure that all internal states were instance-local, eliminating singleton dependencies to support multi-environment instantiation. A dedicated configuration (CarBehavior_Parallel.yaml) was developed, with hyperparameters such as batch_size and buffer_size scaled by 4x to accommodate the increased data throughput. However, due to observed instabilities in the training dynamics within the parallel configuration, likely attributable to the current hyperparameter scaling and synchronization, this experiment was

discontinued and ultimately excluded from the final results to maintain the consistency and reliability of the reported performance metrics.



6.2 Test

6.2.1. Training Monitoring (Continuous)

During all training runs, TensorBoard and Python Command Line Interface (CLI) was used to monitor Mean Reward and Std of Reward at every 5,000-step interval. These metrics served as the primary real-time performance indicators. A converging Mean Reward with decreasing Std of Reward was taken as the criterion for a successful training stage. For example; CurriculumRun1, convergence was observed at approximately step 280,000 where Mean Reward stabilized at 13.4 with Std dropping below 0.08 and remaining there; indicating a near-deterministic, fully converged policy. Subsequent curriculum stages were monitored for signs of catastrophic forgetting: a sudden collapse in Mean Reward at the start of a new stage would indicate that the fine-tuned network had lost previously acquired skills. No such collapse was observed across any curriculum transition, confirming that the incremental difficulty design was effective.

6.2.2 In-Distribution Inference Test

After each training run, the resulting ONNX model was loaded into Unity in inference-only mode and observed running multiple episodes in the original training scene. This qualitative assessment verified that the policy behaved as intended; the agent steered toward waypoints, avoided obstacles, and did not exhibit degenerate behaviors such as wall-hugging

or spinning in place. The Best Reward metric displayed in the UI provided a quick quantitative summary: a Best Reward close to the theoretical maximum (5 waypoints × 0.3 + finish 2.0 + distance rewards ≈ 13-14) indicated a well-performing model.

6.2.3 Out-of-Distribution Generalization

To assess whether the trained models had learned a generalizable navigation policy or had simply memorized the specific training route, each final model was evaluated in a modified scene: the agent's spawn position and initial heading were altered and the finish line position was relocated. This test is the closest analogue to a system-level validation test available within the simulation-only scope of this project.

Results were as follows. CurriculumRun5 consistently navigated the modified track and approached the finish line across multiple episodes. PPO_Run1 reached approximately the halfway point of the modified track before failing. SAC_Run1 rarely passed the third waypoint. These results demonstrate a clear performance ranking (Curriculum Learning > PPO single-run > SAC) that is consistent across both the training environment and the generalization test, providing evidence that the performance differences reflect genuine policy quality rather than overfitting to the training scene.

6.2.4 Performance Test

Wall-clock training time was recorded for all runs via the Time Elapsed field in the ML-Agents training logs. At the 5,000-step mark, PPO required approximately 42 seconds while SAC required approximately 195 seconds; a ratio of approximately 4.6x. By step 500,000 this gap widened to approximately 6.7x: PPO completed 500,000 steps in approximately 3,462 seconds (58 minutes) while SAC required approximately 23,244 seconds (6.5 hours). This performance difference was attributed to the replay buffer management overhead in SAC's off-policy architecture, which introduces significant Python-Unity gRPC communication latency. Additionally, Unity Editor frame stuttering was observed during SAC training, further confirming that the replay buffer mechanism imposes a practical constraint in this communication architecture.

Ch7: Evaluation

7.1 Understanding the Metrics

Before presenting the results, it is important to clarify the metrics reported by ML-Agents throughout training. Mean Reward refers to the average cumulative reward per episode over the most recent reporting window, logged every 5,000 steps. This is the primary indicator of training progress. Std of Reward is the standard deviation of per-episode rewards over the same window; a high value indicates that the agent's performance is highly variable across episodes, sometimes completing large portions of the track and sometimes failing early. Best Reward is a custom metric displayed in the Unity UI, recording the highest single-episode cumulative reward observed so far, updated only at episode termination. Because Best Reward includes the +2.0 Finish bonus, it can appear higher than Mean Reward during early training when the agent occasionally reaches the finish by chance. As the policy matures and the finish is reached consistently, Best Reward and Mean Reward converge. This behavior is expected and is not an error in the implementation.

7.2 Curriculum Learning Results (CurriculumRun1 to Run5)

CurriculumRun1, trained with no obstacles, produced the clearest learning curve of all runs. Mean Reward rose steadily from approximately -1.0 at step 5,000 to 13.4 at step 280,000, after which it plateaued with Std of Reward dropping below 0.08 and remaining stable. This tight plateau indicates a fully converged, near-deterministic policy: the agent reliably completes the track every episode without any collisions.

CurriculumRun2, trained with static obstacles and initialised from the CurriculumRun1 checkpoint, began at Mean Reward of approximately 0.89 rather than -1.0, demonstrating that the fine-tuned network retained its road-following ability while beginning to adapt to the new task. Convergence to approximately 13.0 was reached at step 135,000, significantly faster than CurriculumRun1 required, validating the benefit of warm-starting from a competent prior policy.

CurriculumRun3 introduced two slow-moving obstacles (speeds 0, 0, 0, 1, 1). Mean Reward stabilized in the range of 12.5 to 12.9 with Std oscillating between approximately 0.6 and 2.7. The agent successfully avoided the slower obstacles in most episodes but occasionally collided, causing variance. Critically, no catastrophic forgetting was observed: the agent

maintained a high reward baseline from the very first step, having retained both road-following and static obstacle avoidance knowledge from previous stages.

CurriculumRun4 (speeds 1, 1, 1, 2, 2) showed the highest variance of the curriculum sequence, with Std frequently reaching 3.5 to 4.8 and Mean Reward fluctuating between 6 and 12.4. The faster obstacles presented a substantially harder challenge, and the policy underwent significant re-optimization during this stage. This was the most difficult transition in the curriculum.

CurriculumRun5 (speeds 1, 1, 2, 3, 3) began with Mean Reward in the range of 6 to 8 and showed a slow but consistent upward trend across the 500,000-step budget. Peak Mean Reward reached 13.31 at step 475,000 with Std of 0.165, indicating episodes of near-perfect course completion. The persistent variance throughout this run reflects the inherent stochasticity of the fastest obstacle configuration, where small timing differences produce significantly different collision outcomes.

7.3 PPO_Run1 vs. CurriculumRun5

PPO_Run1 and CurriculumRun5, as both operate under identical obstacle conditions (speeds 1, 1, 2, 3, 3) but differ in how the policy arrived at that stage.

Metric	PPO_Run1	CurriculumRun5
Initial Mean Reward (step 5k)	-0.997	6.19
Best Mean Reward	11.14 (step 495k)	13.31 (step 475k)
Reward stability	Std ~3-3.5	0.205
Reaches Finish in test?	No	Yes, consistently

PPO_Run1 starts from a random policy facing the full difficulty of the dynamic obstacle environment immediately. It requires approximately 25,000 steps to escape the near-zero reward regime. CurriculumRun5, by contrast, begins with a competent policy inherited from four prior stages, achieving a Mean Reward above 6 from the very first reported step. Although both runs receive the same 500,000-step budget at the final obstacle configuration, CurriculumRun5 reaches a substantially higher peak reward and lower standart deviation of reward and is the only approach that reliably completes the course in the generalization test environment.

7.4 PPO_Run1 vs. SAC_Run1

PPO_Run1 and SAC_Run1 were both trained from scratch under identical obstacle conditions, making this a direct algorithm comparison.

Metric	PPO_Run1	SAC_Run1
Wall-clock at step 5k	~42 s	~195 s
Wall-clock at step 500k	~3,462 s (58 min)	~23,244 s (6.5 hours)
Mean Reward at step 500k	~10.0	~4.4
Best Mean Reward	11.14 (step 495k)	7.83 (step 205k)
Reward stability	Moderate (Std ~3-3.5)	Poor (Std ~2.7 to 4.5)
Unity Editor performance	No issues	Stuttering observed

SAC is approximately 6.7 times slower per training session than PPO. This is a direct consequence of the replay buffer management overhead in SAC's off-policy architecture, which introduces significant latency into the Python-Unity gRPC communication cycle. At equal wall-clock time, PPO processes far more environment interactions and consequently achieves a substantially higher policy quality. SAC's Mean Reward peaks at approximately 7.83 at step 205,000 and subsequently declines and oscillates without further improvement, suggesting the algorithm fails to maintain a stable policy in this dynamic stochastic environment. The presence of early off-policy experience in the replay buffer, collected under a much weaker policy, likely degrades gradient quality as training progresses. PPO, being on-policy, always trains on the most recent behavior and avoids this issue entirely.

7.5 Summary

The table below provides a consolidated overview of all training runs and their key outcomes.

Run	Algorithm	Obstacle Speeds	Init From	Peak Mean Reward	Test Performance
CurriculumRun1	PPO	None	Scratch	13.44	Full track, no obstacles
CurriculumRun2	PPO	0,0,0,0,0 static	Run1	13.06	Full track, static obstacles

CurriculumRun3	PPO	0,0,0,1,1	Run2	13.14	Full track, minor variance
CurriculumRun4	PPO	1,1,1,2,2	Run3	12.38	Partial, high variance
CurriculumRun5	PPO	1,1,2,3,3	Run4	13.31	Full track, generalises
PPO_Run1	PPO	1,1,2,3,3	Scratch	11.14	~50% of track
SAC_Run1	SAC	1,1,2,3,3	Scratch	7.83	Poor, rarely past WP3

Ch8: Impact of the Project & Compliance with the Constraints

8.1 Compliance with the Realistic Constraints

This project was developed by a single student on a personally owned workstation within one semester academic timeline. The primary constraints encountered were computational and temporal rather than financial. Training time was the most significant practical limitation: SAC required approximately 6.5 hours for 500,000 steps, and the full curriculum sequence across five stages required multiple training sessions conducted over several days. Memory constraints were not a significant issue during single-environment PPO training. No out-of-memory errors were encountered. All software used in this project is free or open-source, resulting in zero software expenditure.

Table 8.1 Realistic Constraints & Conditions

Economic Factors	
EXPENDITURES	
Computer	Personally owned workstation (RTX 3080 Ti, i9-11900K, 32 GB RAM). No purchase was made for this project.
Other Devices	NA
Peripherals	NA
Internet Connection	Standard home internet connection used for downloading software packages and Unity Asset Store assets. No cost attributed to the project.
Software	All software is free: Unity Personal Edition, ML-Agents (Apache 2.0 license), Python 3.10.12, PyTorch 2.2.1, TensorBoard. Total software cost: 0
Textbook/Magazine/Support Material	All referenced academic papers were accessed via open-access sources. No purchase required.
Human Resources	Developed entirely by a single student. No paid labor involved.
Other	NA

FUNDING Sources	
University Resources	NA
Project support (SANTEZ, TÜBİTAK, and so on)	NA
Support by the Industry	NA
Self-funded	The project relied on personally owned hardware and free software only. No monetary expenditure was incurred.
Other sources	NA
OTHER CONSTRAINTS	
Memory	GPU VRAM (12 GB) was sufficient for all training configurations. And no memory-related failures occurred.
Runtime efficiency	PPO single-environment training: approximately 58 minutes for 500,000 steps. SAC: approximately 6.5 hours for the same step count. Curriculum Learning required five sequential training sessions. All runs were completed within the project timeline.
MANAGERIAL	
Schedule (time)	Development and training were completed within the Spring semester.

8.2 Impact of the Project

This project demonstrates that a Reinforcement Learning based approach can produce adaptive and generalizable autonomous navigation behavior in a simulated dynamic environment, without any handcrafted movement rules. The primary beneficiaries are researchers and developers working on game AI and simulation-based autonomous systems. From a societal perspective, the project contributes to the growing body of accessible knowledge on practical RL applications, particularly for students and practitioners who wish to understand algorithm selection and curriculum design in dynamic environments.

Table 8.2 Impact Assessment Report

Professional/Ethical Issues	
ETHICS/IT Law	
Copyright in copying multimedia (sound, video, text)	All visual assets were obtained from the Unity Asset Store under the Unity Asset Store End User License Agreement, which permits use in personal and academic projects. No copyright media was reproduced without authorization.
Use of licensed software	All software used is either open-source (ML-Agents: Apache 2.0, PyTorch: BSD license, Python: PSF license) or free for personal and educational use (Unity Personal Edition). No license violations occurred.
Data Privacy	No personal data is collected, stored, or processed by the system. All training data is synthetically generated within the simulation environment.
Use of patented products/ideas	No patented methods or products were incorporated into the system. All algorithms (PPO, SAC) are published in open academic literature.
IT Laws in Turkey (5661 and others)	The system does not involve internet communication, user data, or content distribution. No provisions of Law No. 5651 or related Turkish IT legislation are applicable.
IT Laws - International	NA. The system operates entirely locally with no networked services or data transmission beyond localhost gRPC communication between Unity and the Python trainer.
Plagiarism	All code was written independently by the developer. Referenced algorithms and concepts are cited using the Harvard referencing system throughout this report.
PROFESSIONAL	

Sustainability (use of Licensed and/or open source code)	The project is built entirely on open-source and freely licensed components, ensuring long-term reproducibility without dependency on commercial licenses.
Maintenance	The system requires no ongoing maintenance in its current form. Future extensions would require updated versions of Unity and ML-Agents, which are actively maintained by their respective organizations.
Liability	The system is a simulation only. No real hardware is controlled and no real-world decisions are made by the trained agent. No liability risk exists for the developer, any institution, or any third party.
Financial impact/Manufacturability	The system has no manufacturing cost and requires no dedicated hardware beyond a standard developer workstation. As all software components are open source, the approach could be replicated or extended by other researchers at zero additional cost.
SOCIAL/POLITICAL/ ENVIRONMENTAL	
Political impact	NA. The system has no political implications.
Impact on health	NA. The system does not affect physical or mental health of any user beyond standard screen usage.
Gambling	NA.
Pornography	NA.
Equal Access/equity	The project uses exclusively free and open-source software, making it fully reproducible by any student or researcher with access to a CUDA-capable GPU, without financial barriers.
Environmental impacts (energy, carbon footprint and so on)	The computational requirements of this project are modest in the context of machine learning research. All training was conducted on a single consumer-grade GPU over the course of a student project, without the large-scale distributed

	computing infrastructure typically associated with significant energy consumption in AI research. The carbon footprint of this work is therefore negligible, and no meaningful environmental impact is anticipated.
Technology acceptance & Human/Business psychology	NA. The system is not deployed to end users and requires no change management.
Security issues	No security risks exist. The system operates entirely locally with no external network communication.
PROFESSIONAL (CODES, STANDARDS, FRAMEWORKS)	
IEEE	The project follows general IEEE software engineering principles in its documentation structure, including requirements specification and design documentation consistent with IEEE 830 (SRS) conventions.
ISO	NA. No ISO certification or compliance is required for an academic simulation project.
ANSI	NA.
TSE	NA.
ITIL/COBIT	NA.
OTHER	The ML-Agents toolkit follows Unity Technologies's own published API conventions and versioning standards. PyTorch follows the standard deep learning framework API conventions established in the research community.

Ch9: Conclusions

This project set out to address a well-defined limitation of traditional rule-based vehicle AI systems: their inability to adapt to dynamic, unpredictable environments without manual reconfiguration. Using the Unity ML-Agents framework, a reinforcement learning based autonomous car agent was designed, trained, and evaluated in a custom simulation environment featuring a winding road, ordered waypoints, and five dynamic obstacles of varying speeds. Three training strategies were implemented and compared under controlled conditions: PPO trained from scratch, SAC, and a five-stage Curriculum Learning pipeline using PPO with progressively increasing obstacle difficulty.

The principal findings of the project are as follows:

Curriculum Learning produced the most robust and generalizable policy. By exposing the agent to incrementally harder obstacle configurations across five sequential training stages, each initialized from the previous checkpoint, the agent built upon prior knowledge without suffering catastrophic forgetting. CurriculumRun5 was the only model that consistently reached the finish line in both the original training environment and the modified generalization test scene, demonstrating that the learned policy reflects genuine navigation capability rather than route memorization.

PPO outperformed SAC significantly in this setting. PPO completed 500,000 training steps in approximately 58 minutes and achieved a peak Mean Reward of 11.14, while SAC required approximately 6.5 hours for the same step count and peaked at 7.83. The replay buffer mechanism central to SAC's off-policy architecture introduced substantial computational overhead in the Unity gRPC communication pipeline, making SAC impractical for this type of simulation environment.

Ray Perception Sensors proved to be a sufficient and practical analogue for LiDAR-style sensing in simulation. The 9-ray front and 5-ray rear sensor configuration provided the spatial awareness necessary for collision avoidance and waypoint navigation without the complexity of real sensor hardware.

The contributions of this project extend across both practical simulation engineering and autonomous systems research. Unlike conventional navigation agents often developed within static, low-variance environments, this work deliberately maximizes environmental stochasticity by integrating dynamic obstacles within a restricted visibility field. This design

forces the agent to operate under high entropy, compelling it to develop genuine reactive maneuvers rather than memorized path-following behaviors. Consequently, this study demonstrates that even within the constraints of a single-developer project, it is possible to synthesize RL-based policies that maintain robustness despite significant environmental unpredictability.

Beyond the specific scope of this study, the proposed training pipeline offers a scalable alternative to traditional, rule-based scripting in interactive simulation and game development. By replacing brittle, handcrafted logic with adaptive, curriculum-trained policies, developers can create game AI that dynamically responds to player actions and complex, shifting obstacles. Furthermore, the methodology established here provides a replicable framework for future research in autonomous transportation and mobile robotics, where navigating high-density, dynamic scenes remains a primary technical hurdle.

Appendix A

YAML Configuration Files:

CarBehavior.yaml

```
behaviors:
  CarBehavior:
    trainer_type: ppo
    max_steps: 1000000
    time_horizon: 64
    summary_freq: 5000

    hyperparameters:
      batch_size: 512
      buffer_size: 8192
      learning_rate: 3.0e-4
      beta: 5.0e-3
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 5

    network_settings:
      normalize: true
      hidden_units: 256
      num_layers: 3

    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
```

CarBehavior_SAC.yaml

```
behaviors:
  CarBehavior:
    trainer_type: sac
    max_steps: 1000000
    time_horizon: 64
    summary_freq: 5000

    hyperparameters:
      learning_rate: 3.0e-4
      learning_rate_schedule: constant
      batch_size: 256
      buffer_size: 50000
      buffer_init_steps: 1000
      tau: 0.005
      steps_per_update: 1
      save_replay_buffer: false
      init_entcoef: 1.0
      reward_signal_steps_per_update: 1

    network_settings:
      normalize: true
      hidden_units: 256
      num_layers: 3

    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
```

CarBehavior Parallel.yaml

```
behaviors:
  CarBehavior:
    trainer_type: ppo
    max_steps: 1000000
    time_horizon: 64
    summary_freq: 5000

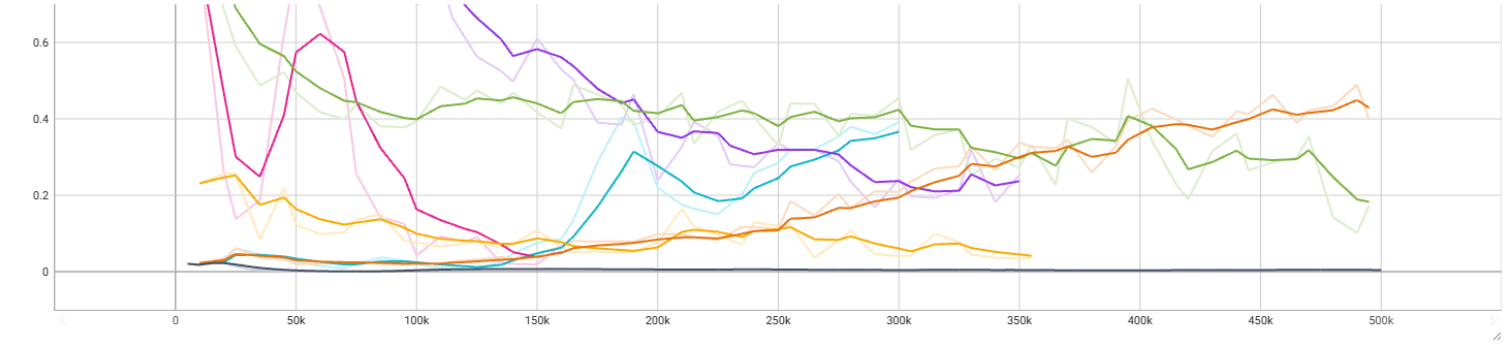
    hyperparameters:
      batch_size: 2048
      buffer_size: 32768
      learning_rate: 3.0e-4
      beta: 5.0e-3
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 5

    network_settings:
      normalize: true
      hidden_units: 256
      num_layers: 3

    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
```

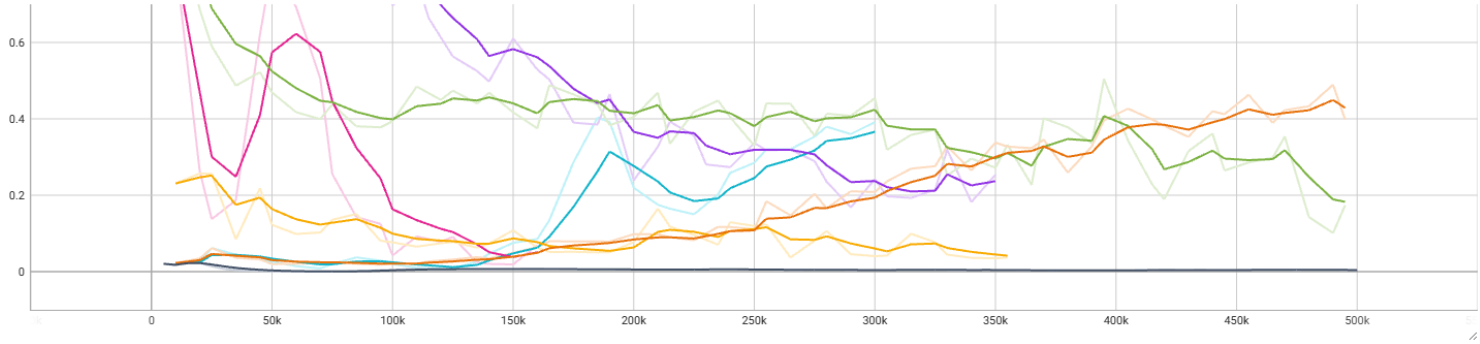
Appendix B

Graphics:



Run	Smoothed	Value	Step	Relative
CurriculumRun1\CarBehavior	0,3661	0,3913	300.000	33.01 min
CurriculumRun2\CarBehavior	0,0505	0,0691	160.000	19.57 min
CurriculumRun3\CarBehavior	0,0417	0,037	355.000	39.58 min
CurriculumRun4\CarBehavior	0,2368	0,2535	350.000	41.29 min
CurriculumRun5\CarBehavior	0,1829	0,1735	495.000	59.45 min
PPO_Run1\CarBehavior	0,4289	0,3984	495.000	55.86 min
SAC_Run1\CarBehavior	0,0043	0,0042	500.000	6.403 hr

Cumulative Reward Graphic



Run	Smoothed	Value	Step	Relative
CurriculumRun1\CarBehavior	0,3661	0,3913	300.000	33.01 min
CurriculumRun2\CarBehavior	0,0505	0,0691	160.000	19.57 min
CurriculumRun3\CarBehavior	0,0417	0,037	355.000	39.58 min
CurriculumRun4\CarBehavior	0,2368	0,2535	350.000	41.29 min
CurriculumRun5\CarBehavior	0,1829	0,1735	495.000	59.45 min
PPO_Run1\CarBehavior	0,4289	0,3984	495.000	55.86 min
SAC_Run1\CarBehavior	0,0043	0,0042	500.000	6.403 hr

Value Loss Graphic



Standard Deviation of Reward

References

- Bengio, Y., Louradour, J., Collobert, R. & Weston, J. (2009). Curriculum learning. *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, pp. 41–48.
- Haarnoja, T., Zhou, A., Abbeel, P. & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning (ICML)*.
- Juliani, A., Berges, V., Teng, E., Cohen, A., Harper, J., Elion, C., Goy, C., Gao, Y., Henry, H., Mattar, M. & Lange, D. (2020). Unity: A general platform for intelligent agents. *arXiv preprint*, arXiv:1809.02627.
- McCloskey, M. & Cohen, N. J. (1989). Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24, pp. 109–165.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint*, arXiv:1707.06347.
- Sutton, R. S. & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- Savid, Y., Mahmoudi, R., Maskeliūnas, R., & Damaševičius, R. (2023). Simulated Autonomous Driving Using Reinforcement Learning: A Comparative Study on Unity's ML-Agents Framework. Information, 14(5), 290. https://doi.org/10.3390/info14050290
- Emuna, R., Borowsky, A. & Biess, A. (2020). Deep reinforcement learning for human-like driving policies in collision avoidance tasks of self-driving cars. *arXiv preprint*, arXiv:2006.04218.
- Raut, U., Galchhaniya, P., Nehete, A., Shinde, R., & Bhoite, A. (2024, July). Unity ML-agents: Revolutionizing gaming through reinforcement learning. In 2024 2nd World Conference on Communication & Computing (WCONF) (pp. 1-7). IEEE.
- Sung, J., Kim, K., & Jo, K. (2025). Reinforcement Learning Autonomous Driving via Reward Function Design in Simulation. 2025 17th International Conference on Human System Interaction (HSI), 1-4.